

PROMO 2025-2026



Rapport de représentation
de données et de
connaissances : Sudoku

DJENNAD Mahdi
RABEHI Milhane
RAPHOZ Timéo
TRUONG Tin

Master 1 MIASHS

M. GENSEL Jérôme

Table des matières :

1.	<u>INTRODUCTION</u>	<u>2</u>
1.A	CONTEXTE & OBJECTIFS	2
1.B	ARCHITECTURE TECHNIQUE	2
2.	<u>MODELISATION ET VALIDATION STRUCTURELLE (DONNEES)</u>	<u>2</u>
2.A	STRUCTURE XML ET CHOIX DE MODELISATION.....	2
2.B	VALIDATION PAR SCHEMA XSD.....	3
3.	<u>LOGIQUE METIER ET DIAGNOSTIC (TRANSFORMATION XSLT).....</u>	<u>4</u>
3.A	DETECTION DES ERREURS (XPATH)	4
3.B	GESTION DES ETATS DU JEU	4
4.	<u>VISUALISATION ET EXPERIENCE UTILISATEUR (SVG)</u>	<u>5</u>
4.A	ERGONOMIE VISUELLE	5
4.B	FEEDBACK VISUEL EN TEMPS REEL	5
5.	<u>FONCTIONNALITE AVANCEE : AIDE A LA RESOLUTION</u>	<u>5</u>
5.A	PRINCIPE ALGORITHMIQUE	5
5.B	MISE EN ŒUVRE.....	6
6.	<u>JEUX D’ESSAI ET VALIDATION</u>	<u>6</u>
7.	<u>CONCLUSION</u>	<u>6</u>

1.Introduction

1.a Contexte & objectifs

Ce projet s'inscrit dans le cadre de l'UE "Représentation des Données et des Connaissances".

L'objectif est de concevoir une chaîne de traitement complète basée sur les technologies XML pour modéliser, valider et visualiser des grilles de Sudoku (9 x 9)

1. Le projet répond à quatre objectifs principaux définis par le cahier des charges :
2. Définir un schéma XSD pour structurer les données
3. Produire des documents XML représentant différents états de la grille
4. Développer une transformation XSLT pour visualiser la grille en SVG et diagnostiquer son état (Correcte, Incorrecte, Gagnante)
5. Implémenter une aide au joueur suggérant les placements possibles pour un chiffre donné

1.b Architecture technique

Nous avons opté pour une architecture respectant le principe de séparation entre le fond (les données) et la forme (la vue). Le navigateur web agit comme moteur de transformation XSLT côté client.



Figure 1 : Architecture technique du projet et flux de données.

2.Modélisation et validation structurelle (Données)

La première étape a consisté à définir un modèle de données robuste. L'enjeu est de garantir l'intégrité de la grille avant tout traitement algorithmique.

2.a Structure XML et Choix de Modélisation

Nous avons choisi une représentation hiérarchique reflétant la géométrie de la grille. L'élément racine <sudoku> contient 9 éléments <ligne>, contenant chacun 9 éléments <case>.

Pour optimiser la sémantique, nous avons distingué la valeur de la position :

- Les coordonnées sont stockées dans des attributs (@l pour la ligne, @c pour la colonne).
- La valeur du chiffre joué est le contenu textuel de la balise.

```
<ligne l="1">
  <case c="1"></case>
  ...
  <case c="3">4</case>
  ...
</ligne>
```

2.b Validation par schéma XSD

Le fichier sudoku.xsd assure la validation syntaxique. Nous avons défini des types personnalisés pour empêcher la saisie de données aberrantes.

- Typage Fort : Le type simple chiffreSudoku restreint les valeurs aux entiers compris entre 1 et 9 inclus via : <xsd:minInclusive value="1"/> et <xsd:maxInclusive value="9"/>.
- Contraintes de Structure : Les attributs minOccurs="9" et maxOccurs="9" garantissent que la grille est toujours un carré de 81 cases.

```
<xsd:simpleType name="chiffreSudoku">
  <xsd:restriction base="xsd:integer">
    <xsd:minInclusive value="1"/>
    <xsd:maxInclusive value="9"/>
  </xsd:restriction>
</xsd:simpleType>

<xsd:simpleType name="indiceSudoku">
  <xsd:restriction base="xsd:integer">
    <xsd:minInclusive value="1"/>
    <xsd:maxInclusive value="9"/>
  </xsd:restriction>
</xsd:simpleType>
```

- Validation des Attributs : L'usage de : use="required" rend la définition des coordonnées obligatoire.

```
<xsd:element name="case" minOccurs="9" maxOccurs="9">
  <xsd:complexType>
    <xsd:simpleContent>
      <xsd:extension base="xsd:string">
        <xsd:attribute name="c" type="indiceSudoku"
use="required"/>
      </xsd:extension>
    </xsd:simpleContent>
  </xsd:complexType>
</xsd:element>
```

3. Logique métier et Diagnostic (Transformation XSLT)

C'est le cœur du projet. Le fichier rendu_diagnostic.xslt ne se contente pas d'afficher la grille, il agit comme un moteur de règles pour valider la partie sémantiquement.

3.a Détection des Erreurs (XPath)

Contrairement à un langage impératif (comme Python), nous utilisons des expressions XPath déclaratives pour détecter les doublons. Pour chaque case non vide, nous vérifions l'unicité du chiffre sur trois axes :

- Ligne : Via l'axe parent (../case).
- Colonne : En comparant l'attribut @c (//case[@c=current()/@c]).
- Région (Bloc 3x3) : C'est la partie la plus complexe. Nous utilisons une formule mathématique basée sur la division entière pour identifier le bloc d'appartenance :

$$\text{Bloc} = \text{floor}((\text{Index} - 1) / 3)$$

Voici l'extrait de code clé permettant de détecter une erreur :

```
<xsl:if test="
    (count(../case[normalize-space(.)=normalize-space(current())]) > 1)
    or
    (count(//case[@c=current()/@c and normalize-space(.)=normalize-
space(current())]) > 1)
    or
    (count(//case[
        normalize-space(.)=normalize-space(current())
        and
        floor((number(../@1)-1) div 3) = floor((number(current()/../@1)-1) div 3)
        and
        floor((number(@c)-1) div 3) = floor((number(current()/@c)-1) div 3)
    ]) > 1)
">X</xsl:if>
```

3.b Gestion des états du jeu

Le système détermine l'état global de la grille via une variable testErreur qui accumule les erreurs trouvées.

- Incorrecte : Si un conflit est détecté (variable non vide).
- Gagnante : Si la grille est pleine (not(../case[normalize-space(.)=""])) ET sans erreur.
- Correcte : État par défaut (partie en cours sans conflit).

4. Visualisation et expérience utilisateur (SVG)

La transformation génère un document SVG (Scalable Vector Graphics) pour un rendu graphique vectoriel de haute qualité.

4.a Ergonomie visuelle

Pour améliorer la lisibilité (Experience Utilisateur), nous avons :

- Dessiné un damier (alternance de fonds blancs et gris) pour distinguer visuellement les 9 régions.
- Utilisé des traits épais pour les bordures de régions et fins pour les cellules.

4.b Feedback visuel en temps réel

Le système informe immédiatement le joueur de la validité de ses coups.

- Texte de diagnostic : Affiche l'état de la partie en haut de la grille.
- Marquage des erreurs : Si une case viole une règle du Sudoku, le chiffre s'affiche automatiquement en ROUGE.

Diagnostic : **incorrecte**

		4	6		8			2
6				9	5		4	8
1	9					5		
			7	6		4	2	
4		6	8			7	7	
7		3				8		
9				3	7		8	4
	8	7	4				3	
			2			1		

Figure 2 : Détection visuelle d'un doublon (Diagnostic : Incorrecte)

5. Fonctionnalité avancée : Aide à la résolution

Pour répondre à l'objectif 4, nous avons implémenté un système de suggestions ("Candidats").

5.a Principe Algorithmique

L'algorithme inverse la logique de validation. Au lieu de vérifier si un chiffre présent est faux, il vérifie pour chaque case *vide* si un chiffre candidat (ex: 4) *pourrait* être accepté. Si le chiffre n'est présent ni dans la ligne, ni dans la colonne, ni dans le bloc, la case est surlignée en vert (ou gris).

5.b Mise en œuvre

Nous avons produit des feuilles de style dédiées (ou paramétrables) pour chaque chiffre de 1 à 9.

Placement possible du chiffre 1

		4	6		8			2
6				9	5		4	8
1	9					5		
			7	6		4	2	
4		6	8			7		
7		3				8		
9				3	7		8	4
	8	7	4				3	
			2			1		

Figure 3 :Visualisation des placements possibles pour le chiffre 1.

6.Jeux d'essai et validation

Conformément aux attentes, nous fournissons un jeu de fichiers XML exhaustif testant tous les cas limites :

- `correcte.xml` : Une grille partiellement remplie sans erreur. Sert à valider l'affichage nominal.
- `incorrecte.xml` : Contient des doublons volontaires (ex: deux '5' sur la ligne 1). Valide la coloration rouge et le message "Incorrecte".
- `gagnante.xml` : Une grille complète et valide. Déclenche le message "Gagnante".
- `vide.xml` : Une grille vierge. Valide la robustesse du code face à l'absence de données.
- `test_possible_4.xml` : Démontre la fonctionnalité d'aide au joueur.

7.Conclusion

Ce projet nous a permis de mettre en œuvre une chaîne XML complète. Nous avons répondu à l'ensemble des objectifs fixés par le sujet. La principale difficulté technique a résidé dans la traduction de la logique géométrique du Sudoku (les blocs 3 x 3) en expressions XPath. Nous avons démontré qu'il est possible de créer une application interactive et robuste (validation de types, moteur de règles) en utilisant exclusivement des technologies déclaratives (XSLT/SVG), sans recours à un langage de programmation impératif classique.